Advance Encryption Standard

Dr. Amjad Hussain Zahid

"It seems very simple."

"It is very simple. But if you don't know what the key is it's virtually indecipherable."

—Talking to Strange Men, Ruth Rendell

Origins

- A replacement for DES was needed
  - Key size is too small in DES
- Can use Triple-DES
  - But Triple-DES is slow and uses small plaintext block
- US NIST issued call for ciphers in 1997
  - 15 candidate ciphers accepted in Jun 1998
    - 5 ciphers were shortlisted in Aug 1999

- AES is the result of a **three year** competition.
- This competition was announced in September 1997 and had entries from 12 different countries



AES : Introduction

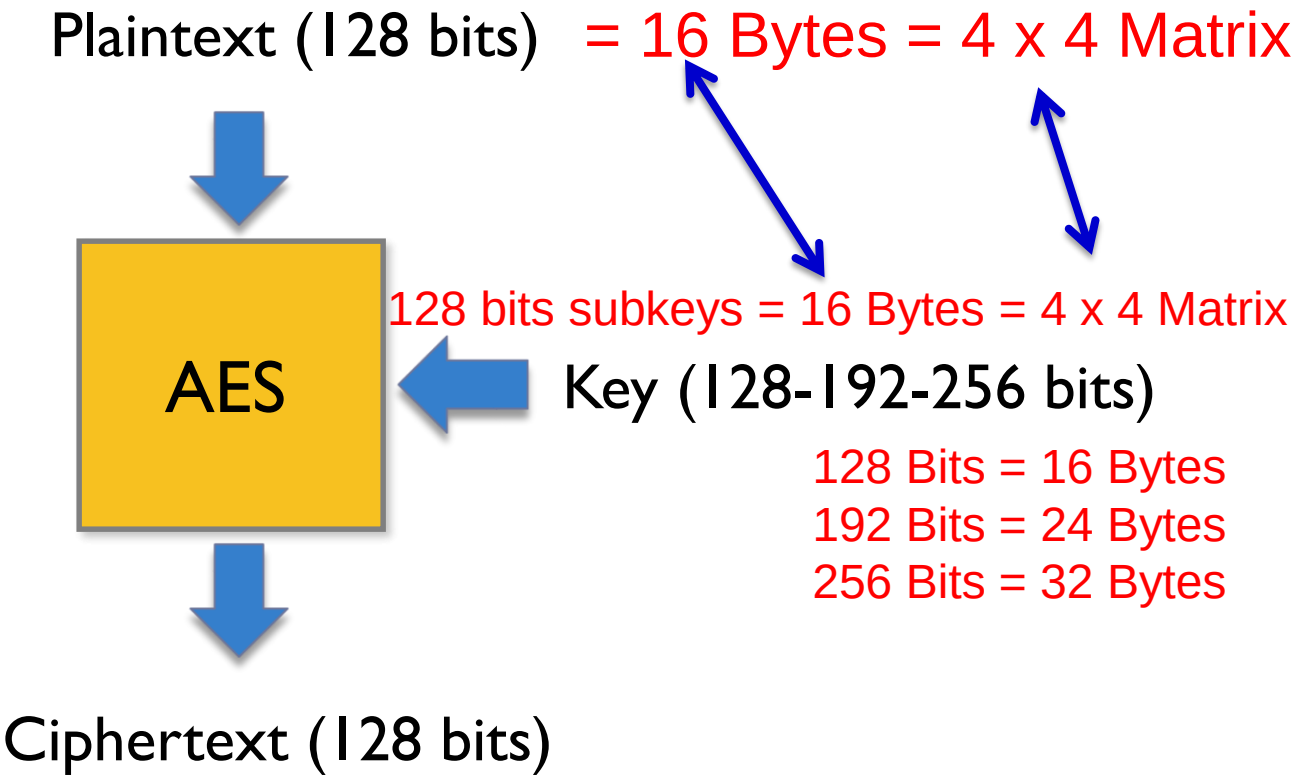
- Private key symmetric block cipher
- 128-bit data
- Three options for key : 128/192/256-bit keys
  - 128 bits (= 3.4×10^{38} keys)
  - 192 bits (= 6.2×10^{57} keys)
  - 256 bits (= 1.1×10^{77} keys)
    - DES uses only 56-bit keys, giving a key space of 7.2×10^{16} keys

8 bits = 1 Byte
4 Bytes = 1 Word

Key size (words/bytes/bits)	4/16/128	6/24/192	8/32/256
Number of rounds	10	12	14
Expanded key size (words/byte)	44/176	52/208	60/240

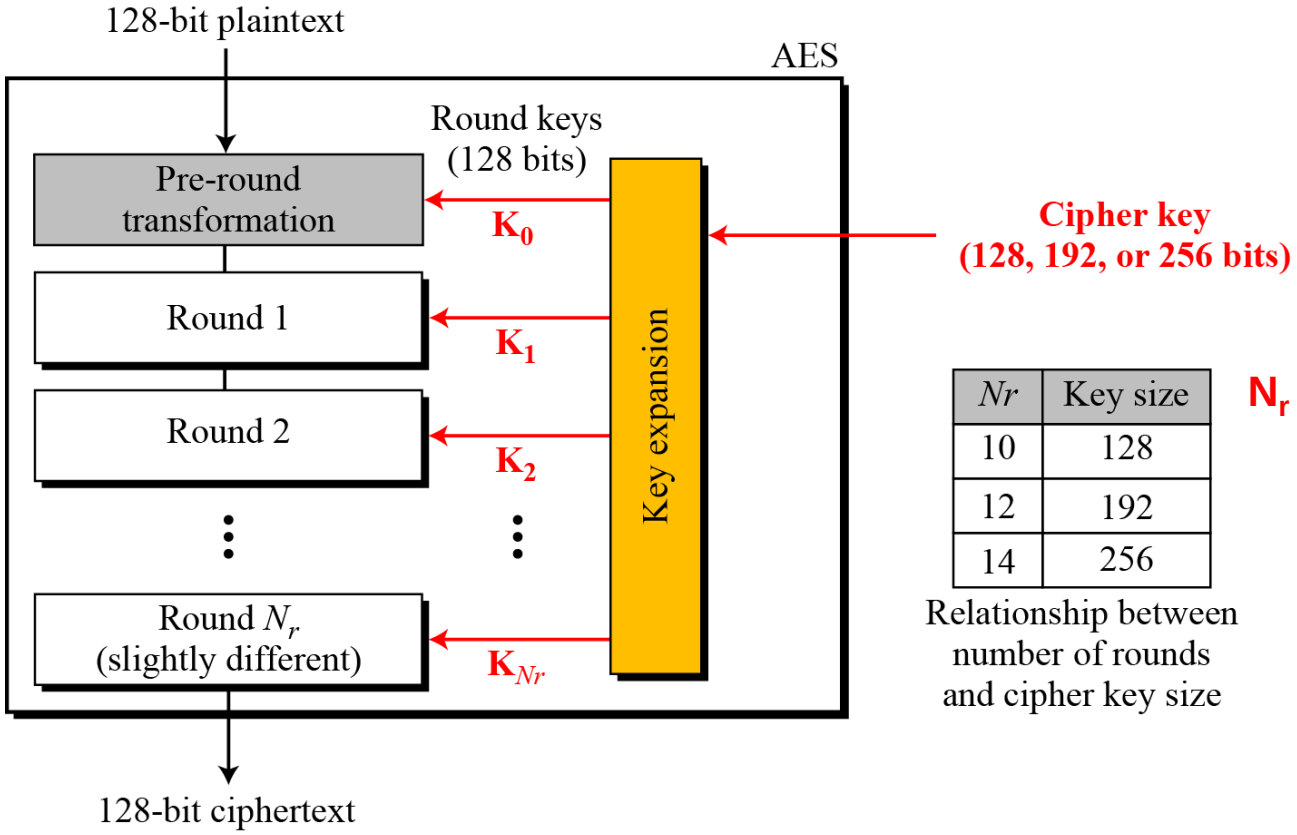


AES Conceptual Scheme



Multiple rounds

- Rounds are (almost) identical
  - First and last round are a little different



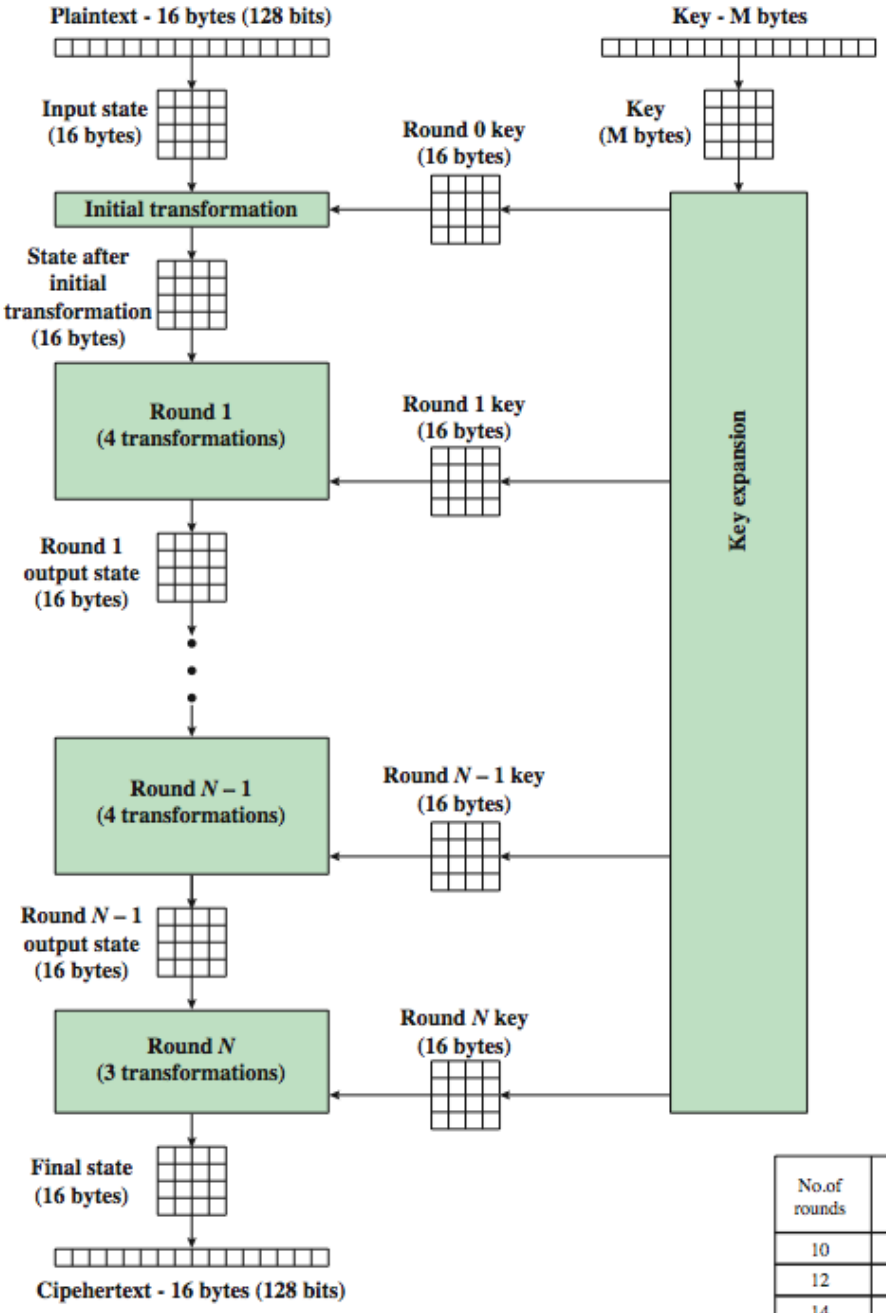
Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

N_r = Number of Rounds

Overall Structure

- An **iterative** rather than **Feistel** cipher having simple structure
- Key expanded into array of 32-bit words: four words form round key in each round
- 4 different steps are used
- only AddRoundKey uses key



No.of rounds	Key Length (bytes)
10	16
12	24
14	32

High Level Description

Key Expansion

- Round keys are derived from the cipher key using Rijndael's key schedule

Initial Round

- AddRoundKey : Each byte of the state is combined with the round key using bitwise xor

Rounds

- SubBytes : non-linear substitution step
- ShiftRows : transposition step
- MixColumns : mixing operation of each column.
- AddRoundKey

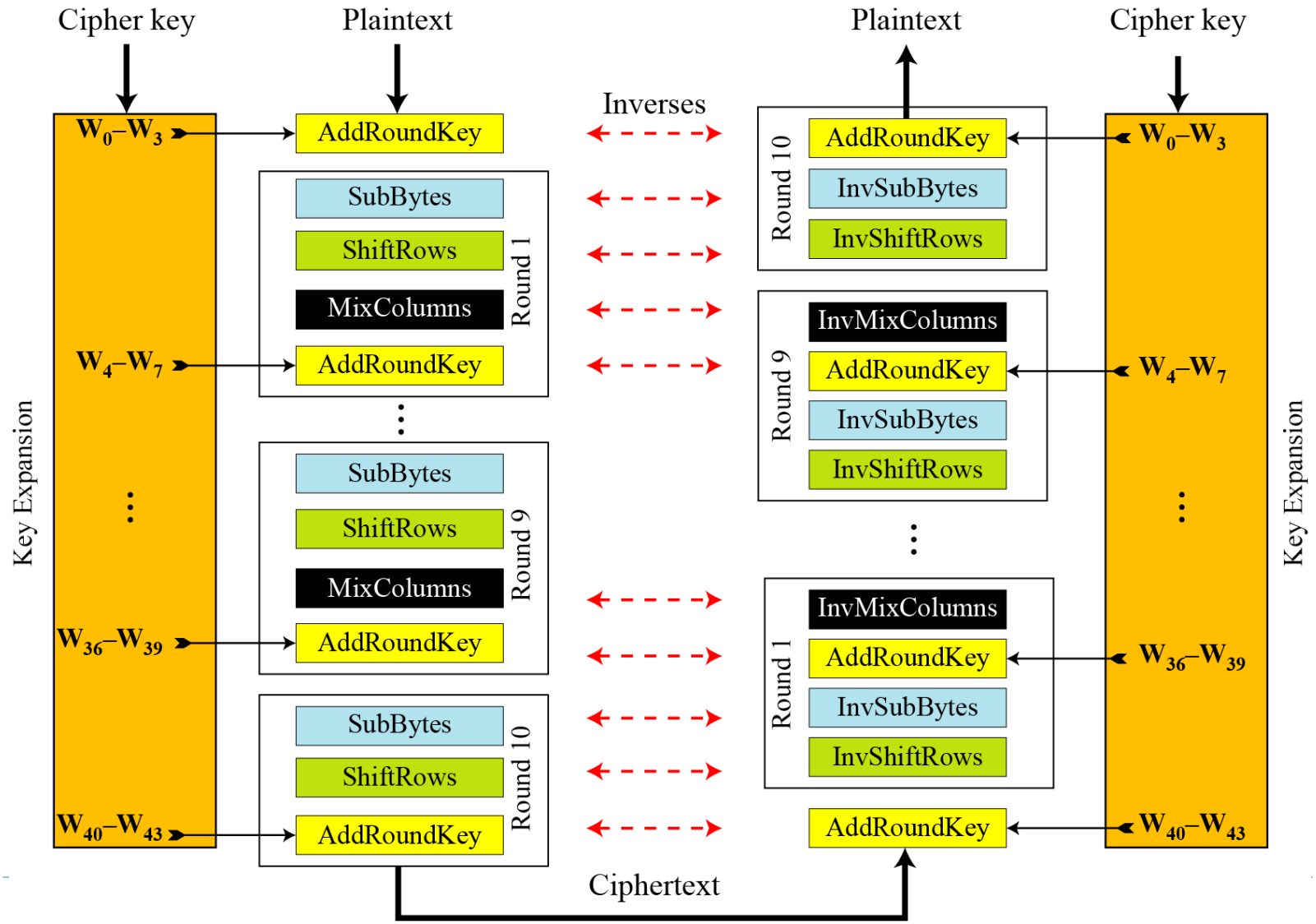
Final Round

- SubBytes
- ShiftRows
- AddRoundKey

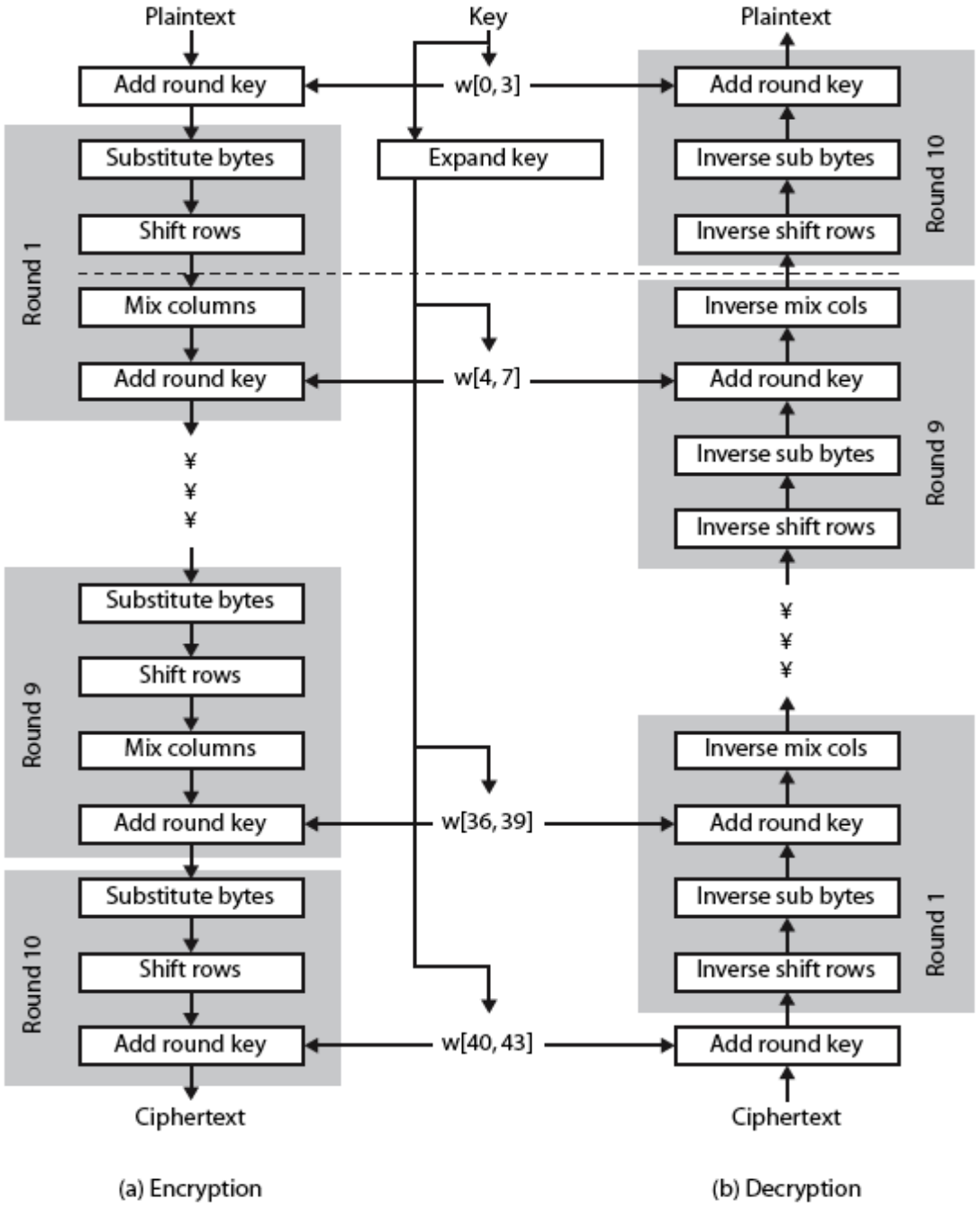
MixColumns step is not included in this round



Overall Structure



Overall Structure



Overall Structure

Fixed Size

Variable Size

- Key received as input array of 4 rows and N_k columns
  - N_k = 4, 6, or 8, parameter which depends on key size
- Input key is expanded into an array of 44/52/60 words
  - Each word has 32 bits (4 Bytes)
- 4 different words serve as a key for each round

k_0	k_4	k_8	k_{12}
k_1	k_5	k_9	k_{13}
k_2	k_6	k_{10}	k_{14}
k_3	k_7	k_{11}	k_{15}

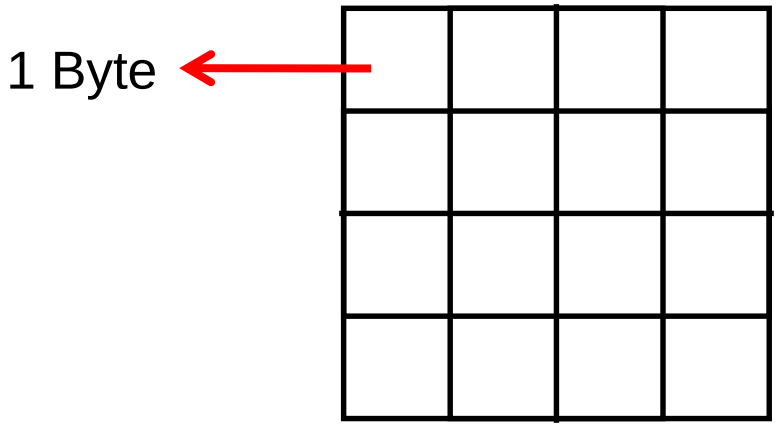
Here, k represents byte.

w_0	w_1	w_2	.....	w_{42}	w_{43}

$$w_0 = \begin{bmatrix} k_0 \\ k_1 \\ k_2 \\ k_3 \end{bmatrix} \quad w_1 = \begin{bmatrix} k_4 \\ k_5 \\ k_6 \\ k_7 \end{bmatrix} \quad w_2 = \begin{bmatrix} k_8 \\ k_9 \\ k_{10} \\ k_{11} \end{bmatrix} \quad w_3 = \begin{bmatrix} k_{12} \\ k_{13} \\ k_{14} \\ k_{15} \end{bmatrix}$$

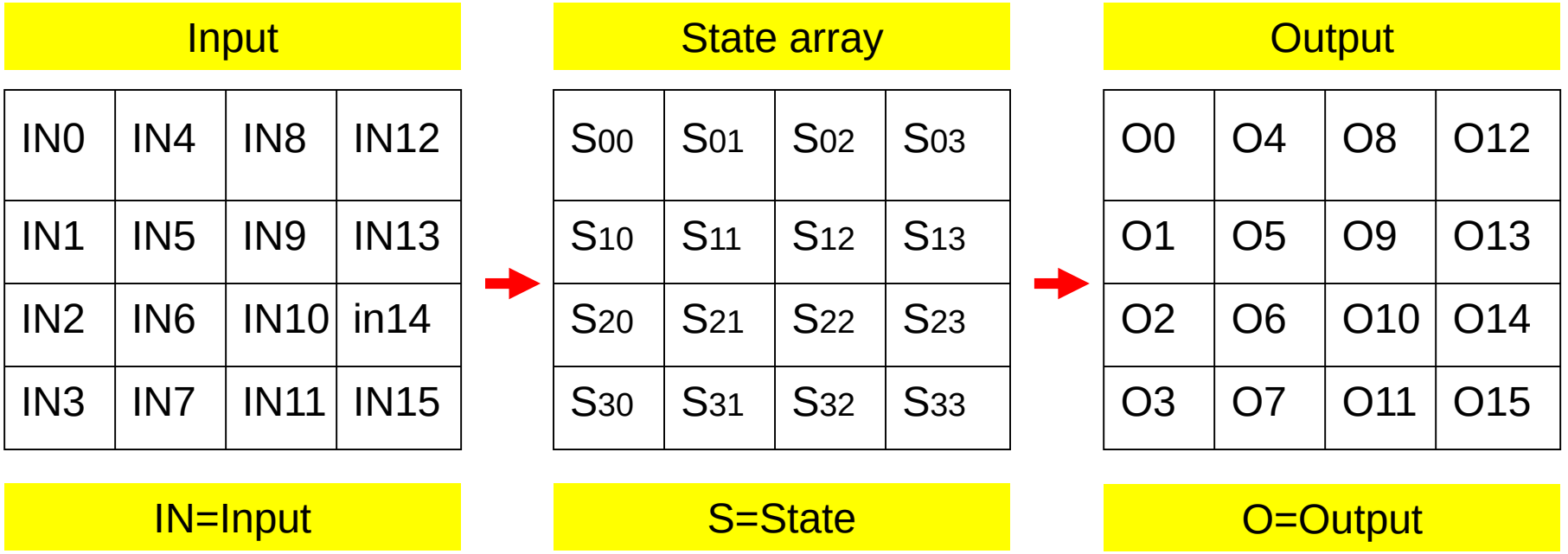
128-bit values

- Data block viewed as 4-by-4 table of bytes
- Represented as 4 by 4 matrix of 8-bit bytes.
- Key is expanded to array of 32 bits words

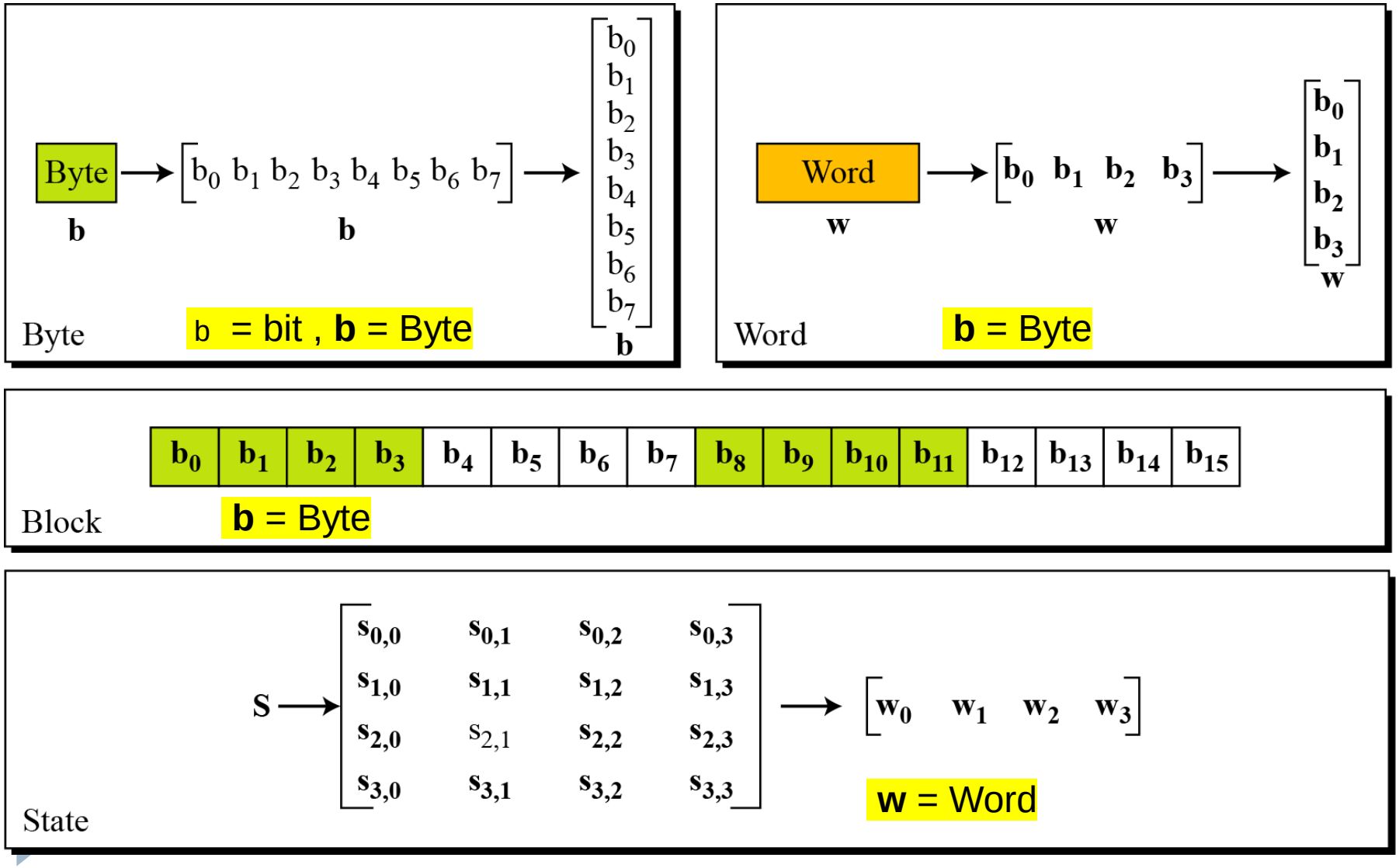


128-bit values

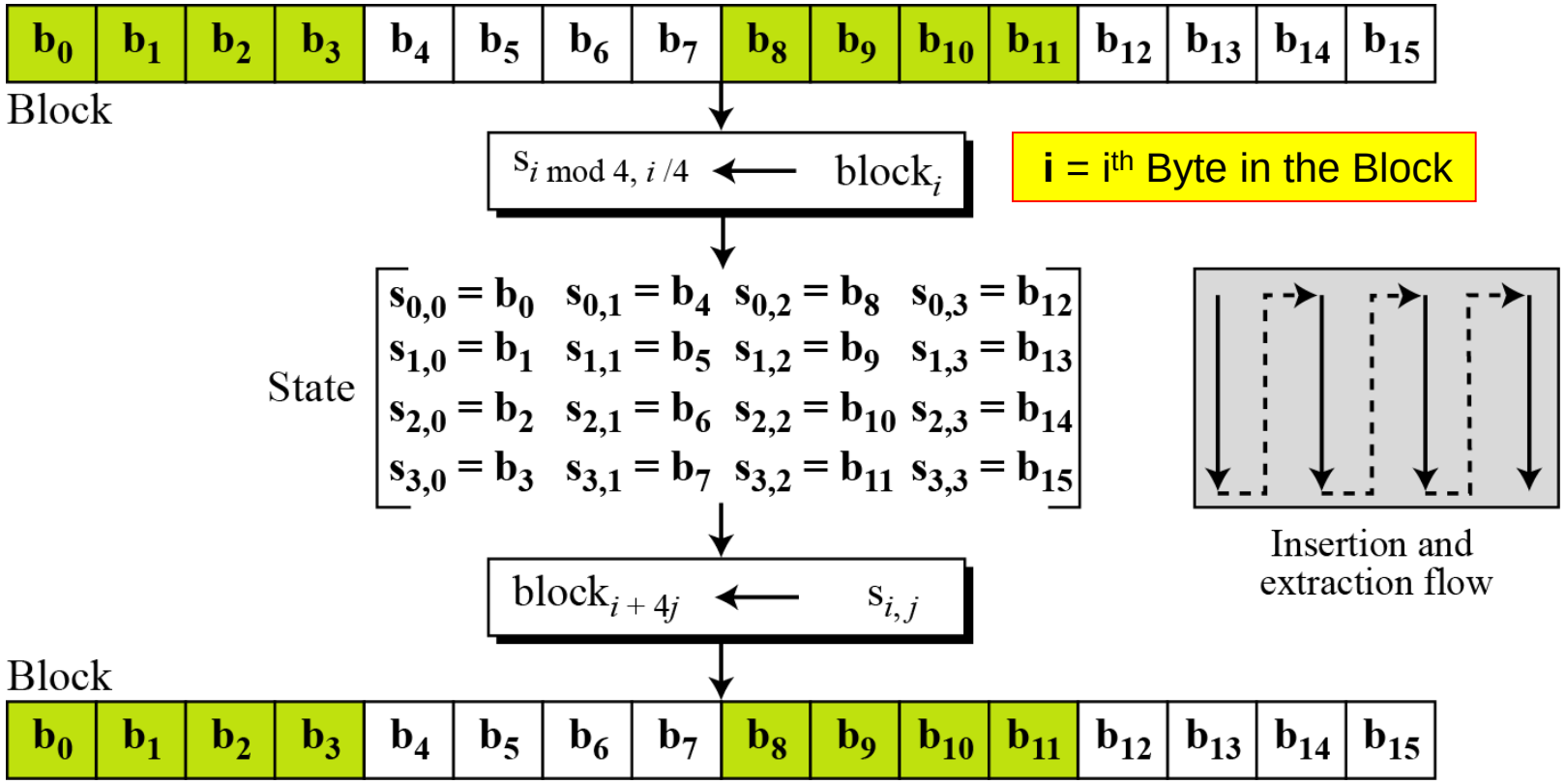
- Single 128 bit block as input
- Copied to a State array with Nb columns (Nb=4)



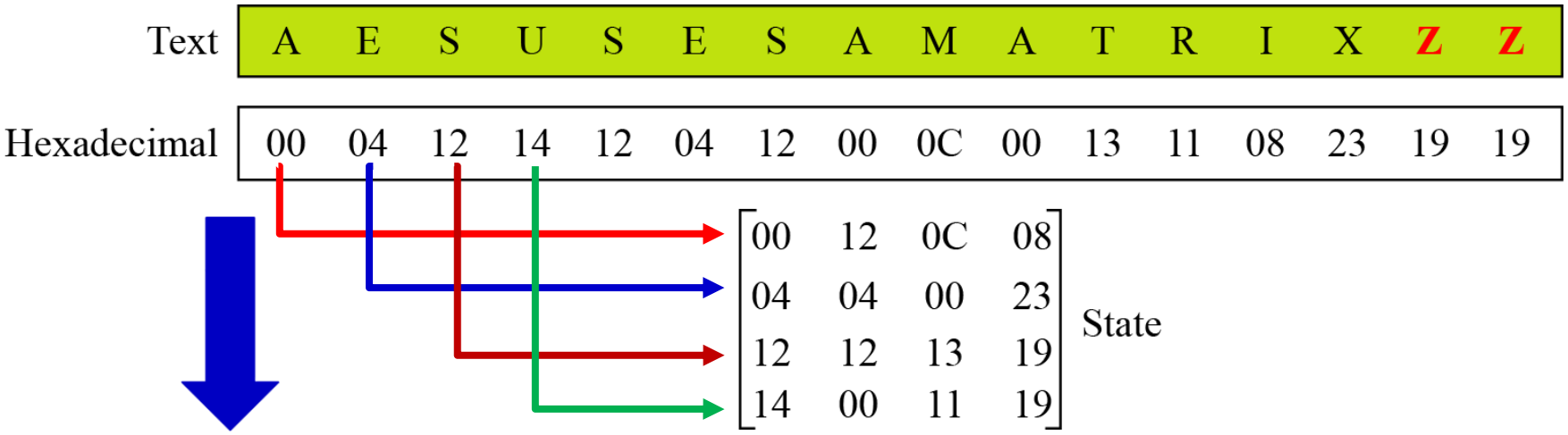
Data Unit


Byte
b
b0 b1 b2 b3 b4 b5 b6 b7
b
b0 b1 b2 b3 b4 b5 b6 b7
b
Byte
b = bit , b = Byte
Word
w
b0 b1 b2 b3
w
b0 b1 b2 b3
w
Word
b = Byte
b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
Block
b = Byte
S
s0,0 s0,1 s0,2 s0,3
s1,0 s1,1 s1,2 s1,3
s2,0 s2,1 s2,2 s2,3
s3,0 s3,1 s3,2 s3,3
w0 w1 w2 w3
w = Word
State

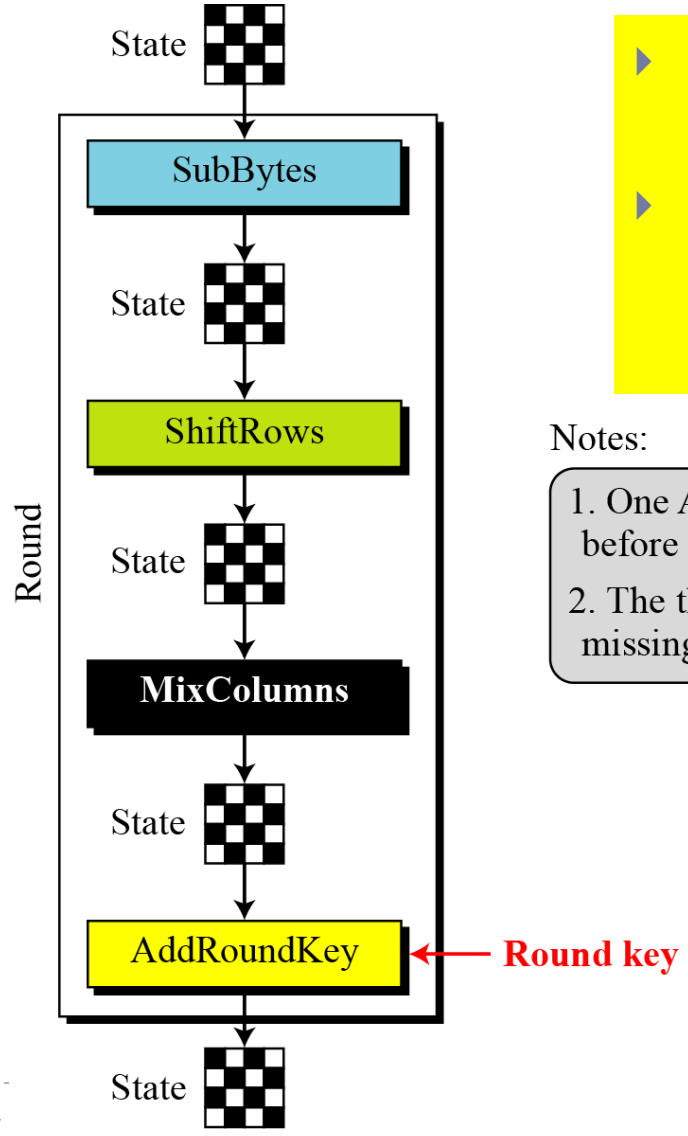
Unit Transformation



Changing Plaintext to State



Details of Each Round



- Each round is a repetition of functions that perform a transformation over State array
- Consists of 4 main functions: one permutation and three substitutions

  Substitute bytes, Shift rows, Mix columns, Add round key

Notes:

1. One AddRoundKey is applied before the first round.
2. The third transformation is missing in the last round.

AES parameters

- Nb – Number of columns in the State
  - For AES, Nb = 4
- Nk – Number of 32-bit words in the Key
  - For AES, Nk = 4, 6, or 8
- Nr – Number of rounds (function of Nb and Nk)
  - For AES, Nr = 10, 12, or 14



Finite Field Arithmetic

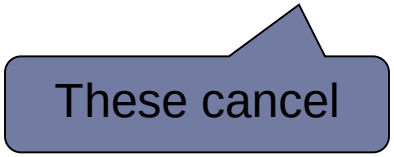
- Addition (XOR)
  - $(x^6 + x^4 + x^2 + x + 1) + (x^7 + x + 1) = x^7 + x^6 + x^4 + x^2$
  - $\{01010111\} \oplus \{10000011\} = \{11010100\}$
  - $\{57\} \oplus \{83\} = \{d4\}$
- Multiplication is tricky



Finite Field Multiplication (•)

$(x^6 + x^4 + x^2 + x + 1)\ (x^7 + x + 1) =$

$x^{13} + x^{11} + x^9 + x^8 + x^7 + x^7 + x^5 + x^3 + x^2 + x + x^6 + x^4 + x^2 + x + 1$

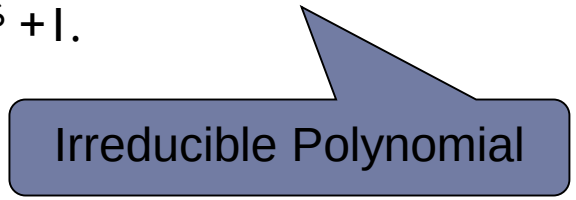


$= x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$

and

$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$ modulo $(x^8 + x^4 + x^3 + x + 1)$

$= x^7 + x^6 + 1.$





SubBytes: Byte Substitution

- A simple substitution of each byte
  - provide a confusion
- Uses one S-box of 16x16 bytes containing a permutation of all 256 8-bit values
- Each byte of state is replaced by byte indexed by row (left 4-bits) & column (right 4-bits)
  - eg. byte {95} is replaced by byte in row 9 column 5
  - which has value {2A}
- S-box constructed using defined transformation of values in Galois Field- $GF(2^8)$
- S-box constructed using a simple math formula using a non-linear function : 1/x. It is the Multiplicative Inverse of x.

Galois : pronounce “Gal-Wa”



Substitution Box

- S-box is generated by determining the multiplicative inverse for a given number in the Galois field, zero would be set to zero



Substitution Box

How to calculate the Multiplicative inverse?

Calculate Multiplicative Inverse of x^6+x^4+x+1 w.r.t. $x^8+x^4+x^3+x+1$.

i	remainder[i]	quotient[i]	auxiliary[i]
1	$x^8 + x^4 + x^3 + x + 1$	+	0
2	$x^6 + x^4 + x + 1$	*	1
3	x^2	$x^2 + 1$	$x^2 + 1$
4	$x + 1$	$x^4 + x^2$	$x^6 + \cancel{x^4} + \cancel{x^4} + x^2 + 1$
5	1	$x + 1$	$x^7 + x^6 + x^3 + \cancel{x^2} + \cancel{x^2} + x + \cancel{1} + \cancel{1}$

Addition in a Finite Field is XOR.
Thus the inverse is $x^7+x^6+x^3+x$ = CA in hexadecimal.



Substitution Box

Multiplicative inverse of 1 is 1

Multiplicative inverse of 2

2 In Binary = 0 0 0 0 0 0 1 0
= X^1
= X

Calculate Multiplicative Inverse of x w.r.t. $x^8+x^4+x^3+x+1$.

Remainder	Quotient	Auxiliary
$x^8+x^4+x^3+x+1$		0
x		1
1	$x^7+x^3+x^2+1$	$x^7+x^3+x^2+1$

Thus the inverse of 2 is $x^7+x^3+x^2+1$ = 10001101 = 8D in hexadecimal.



Substitution Box

Multiplicative inverse of 3

3 In Binary = 0 0 0 0 0 0 1 1
= X^1 1
= X + 1

Calculate Multiplicative Inverse of x + 1 w.r.t. $x^8+x^4+x^3+x+1$.

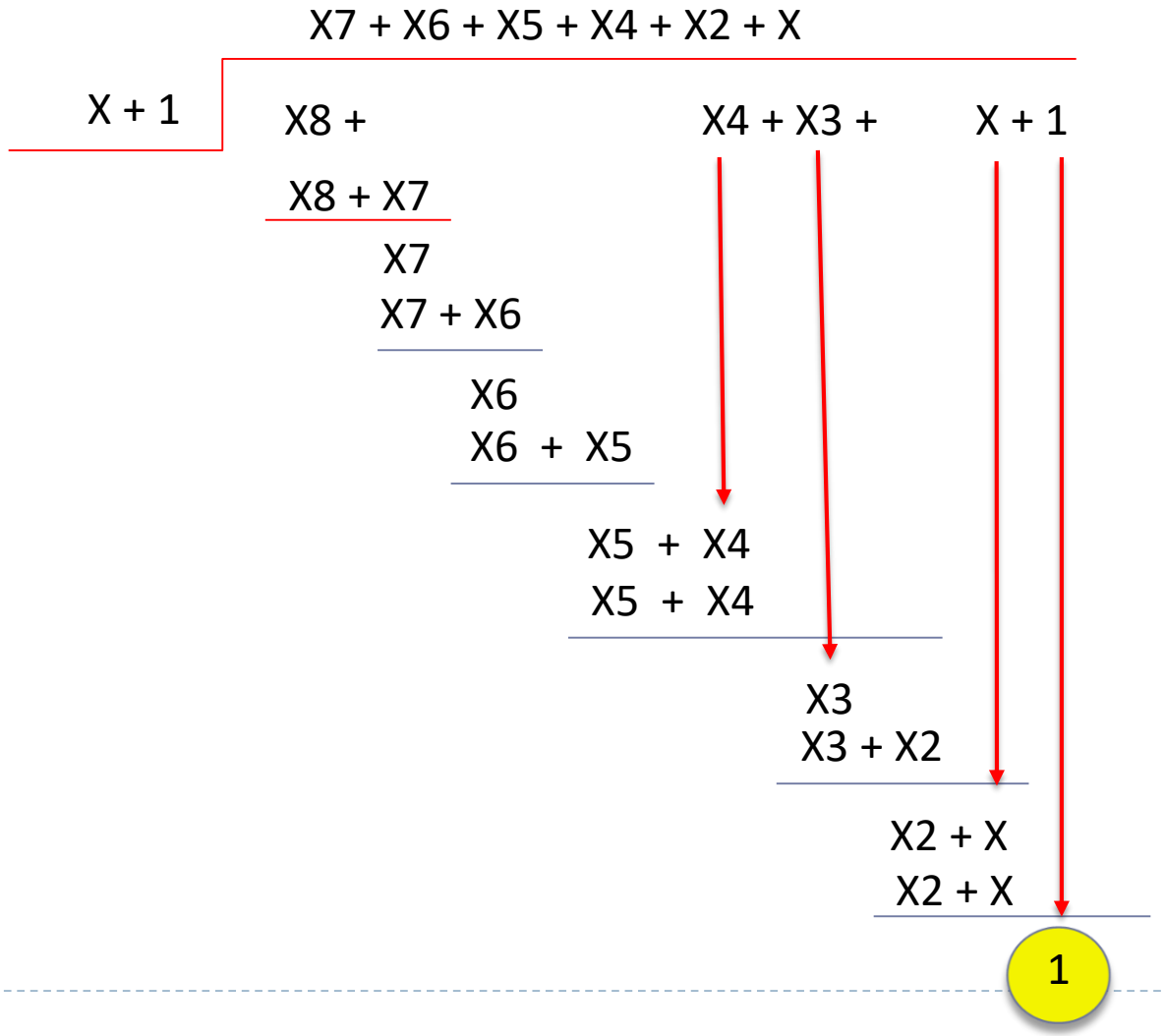
Remainder	Quotient	Auxiliary
$x^8+x^4+x^3+x+1$		0
x + 1		1
1	$x^7+x^6+x^5+x^4+x^2+x$	$x^7+x^6+x^5+x^4+x^2+x$

Thus the inverse of 3 is $x^7+x^6+x^5+x^4+x^2+x$ = 11110110 = F6 in hexadecimal.



Substitution Box

Multiplicative inverse of 3 = x + 1 w.r.t. $x^8+x^4+x^3+x+1$.



Substitution Box

Table of Multiplicative inverses

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	--	01	8d	f6	cb	52	7b	d1	e8	4f	29	c0	b0	e1	e5	c7
1	74	b4	aa	4b	99	2b	60	5f	58	3f	fd	cc	ff	40	ee	b2
2	3a	6e	5a	f1	55	4d	a8	c9	c1	0a	98	15	30	44	a2	c2
3	2c	45	92	6c	f3	39	66	42	f2	35	20	6f	77	bb	59	19
4	1d	fe	37	67	2d	31	f5	69	a7	64	ab	13	54	25	e9	09
5	ed	5c	05	ca	4c	24	87	bf	18	3e	22	f0	51	ec	61	17
6	16	5e	af	d3	49	a6	36	43	f4	47	91	df	33	93	21	3b
7	79	b7	97	85	10	b5	ba	3c	b6	70	d0	06	a1	fa	81	82
8	83	7e	7f	80	96	73	be	56	9b	9e	95	d9	f7	02	b9	a4
9	de	6a	32	6d	d8	8a	84	72	2a	14	9f	88	f9	dc	89	9a
a	fb	7c	2e	c3	8f	b8	65	48	26	c8	12	4a	ce	e7	d2	62
b	0c	e0	1f	ef	11	75	78	71	a5	8e	76	3d	bd	bc	86	57
c	0b	28	2f	a3	da	d4	e4	0f	a9	27	53	04	1b	fc	ac	e6
d	7a	07	ae	63	c5	db	e2	ea	94	8b	c4	d5	9d	f8	90	6b
e	b1	0d	d6	eb	c6	0e	cf	ad	08	4e	d7	e3	5d	50	1e	b3
f	5b	23	38	34	68	46	03	8c	dd	9c	7d	a0	cd	1a	41	1c



SubBytes and InvSubBytes

The multiplicative inverse is transformed using the following affine transformation:

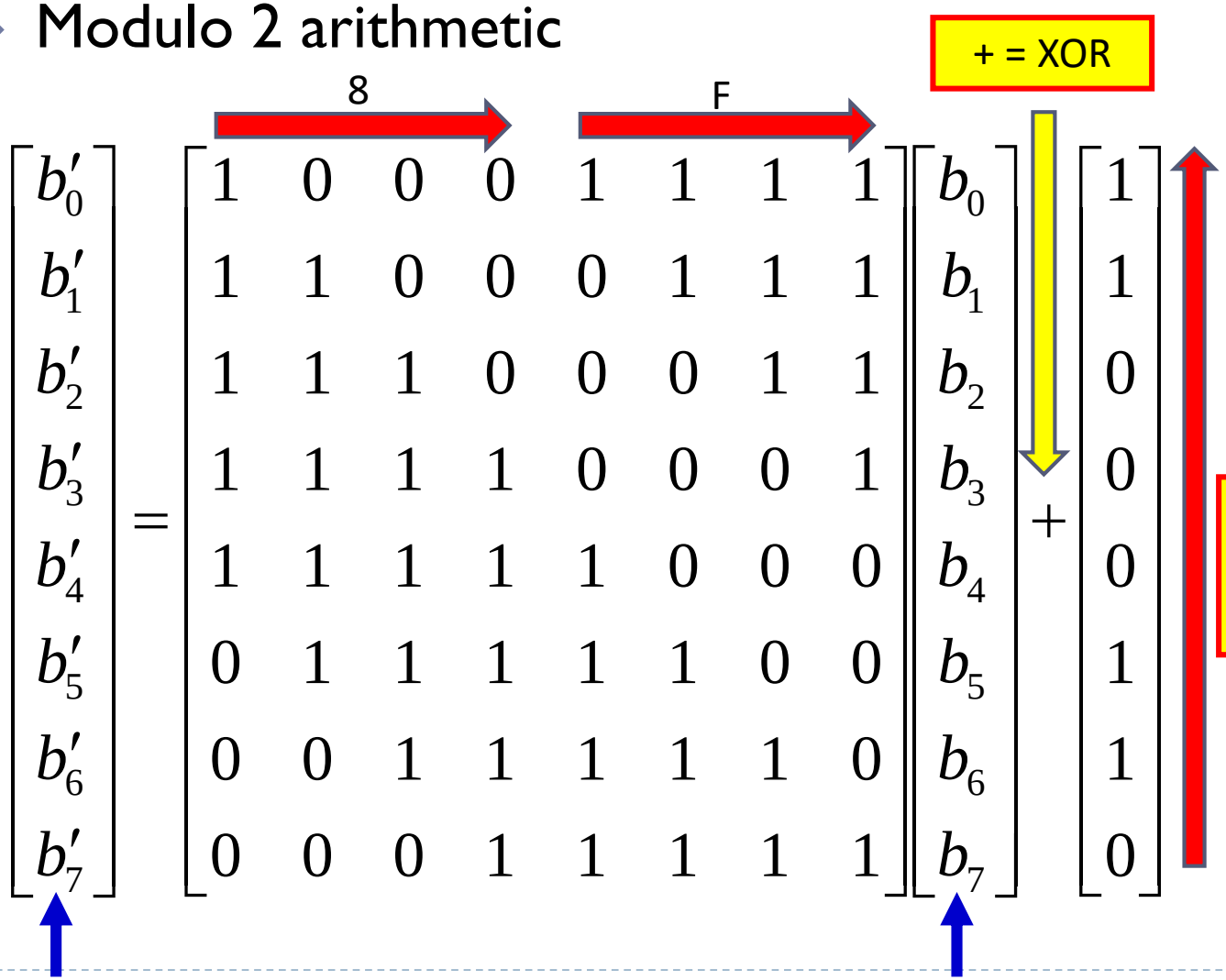
$$\begin{bmatrix} 1&0&0&0&1&1&1&1 \\ 1&1&0&0&0&1&1&1 \\ 1&1&1&0&0&0&1&1 \\ 1&1&1&1&0&0&0&1 \\ 1&1&1&1&1&0&0&0 \\ 0&1&1&1&1&1&0&0 \\ 0&0&1&1&1&1&1&0 \\ 0&0&0&1&1&1&1&1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

where $[x_0, ..., x_7]$ is the multiplicative inverse as a vector.

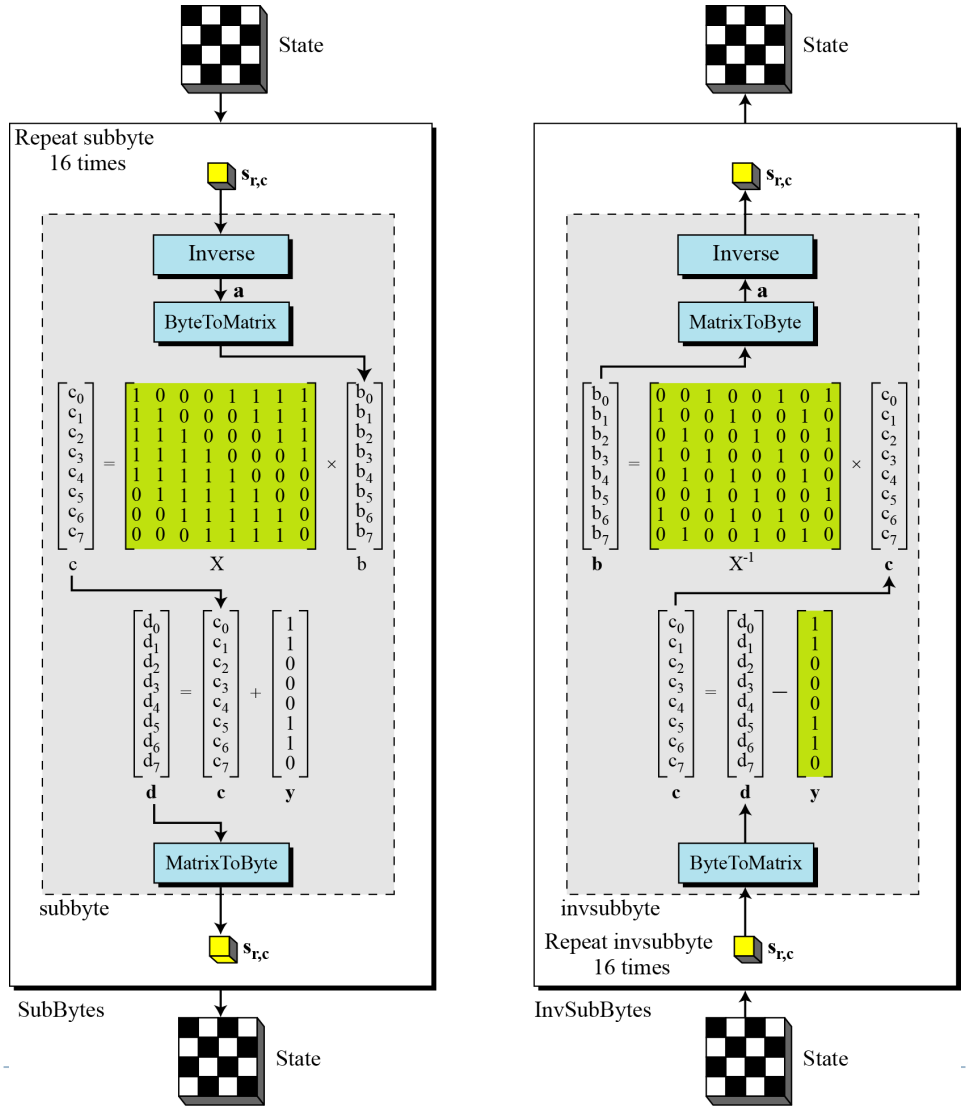


SubBytes and InvSubBytes

- Modulo 2 arithmetic

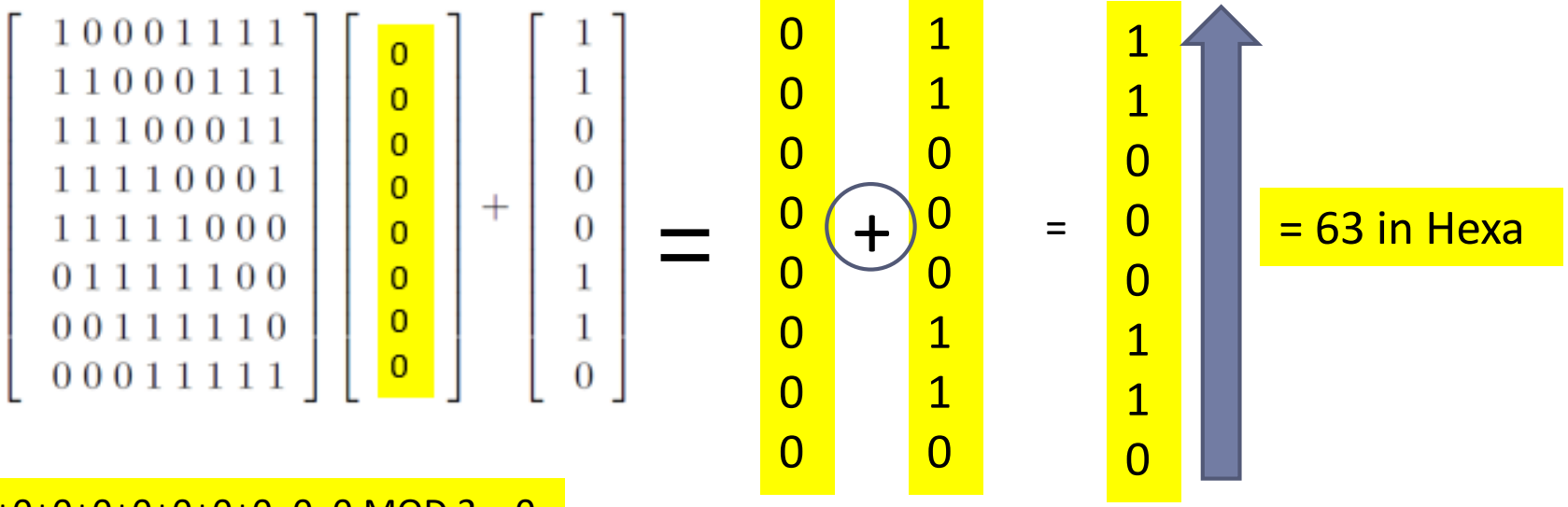


SubBytes and InvSubBytes



SubBytes and InvSubBytes

- For 0



0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

0+0+0+0+0+0+0+0=0=0 MOD 2 = 0

SubBytes and InvSubBytes

- For 1

$$\begin{bmatrix} 1&0&0&0&1&1&1&1 \\ 1&1&0&0&0&1&1&1 \\ 1&1&1&0&0&0&1&1 \\ 1&1&1&1&0&0&0&1 \\ 1&1&1&1&1&0&0&0 \\ 0&1&1&1&1&1&0&0 \\ 0&0&1&1&1&1&1&0 \\ 0&0&0&1&1&1&1&1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{matrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{matrix} \oplus \begin{matrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{matrix} = \begin{matrix} 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \end{matrix} = 7C$$



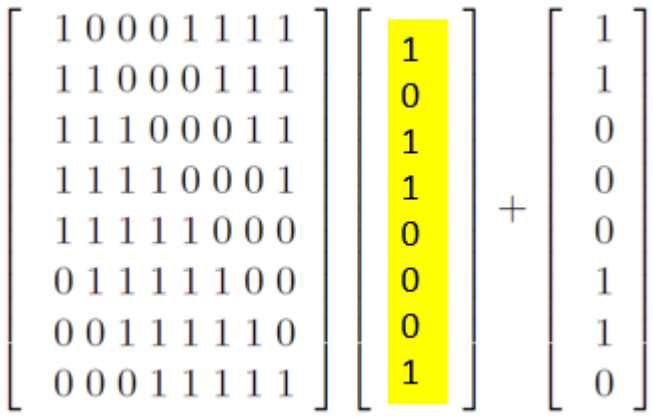
SubBytes and InvSubBytes

- For 2

Inverse of 02 = 10001101 =

Written in reverse

1
0
1
1
0
0
0
1



1+0+0+0+0+0+0+1=2=2 MOD 2 = 0

1+0+0+0+0+0+0+1=2=2 MOD 2 = 0

1+0+1+0+0+0+0+1=3=3 MOD 2 = 1

1+0+1+1+0+0+0+1=4=4 MOD 2 = 0

1+0+1+1+0+0+0+0=3=3 MOD 2 = 1

0+0+1+1+0+0+0+0=2=2 MOD 2 = 0

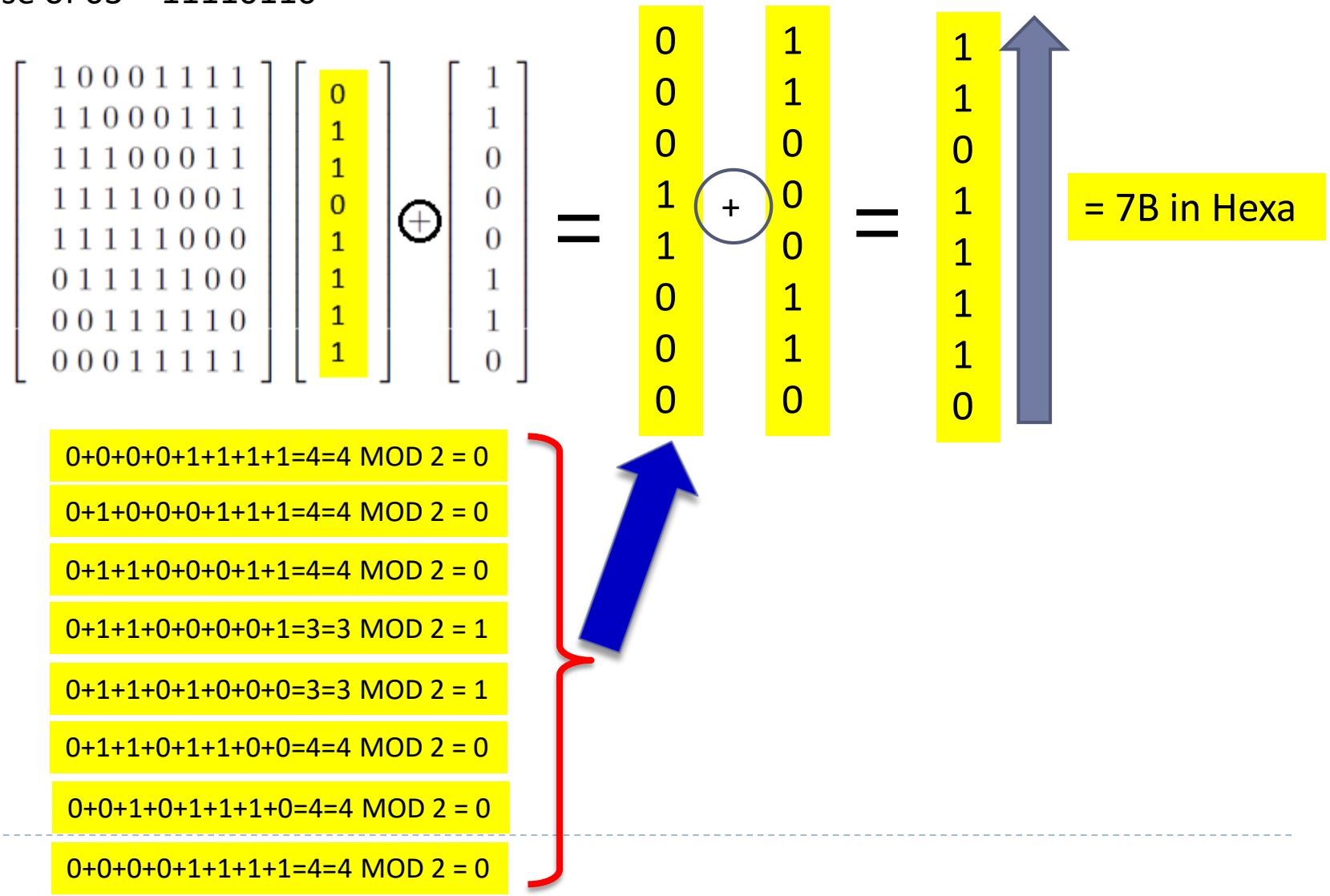
0+0+1+1+0+0+0+0=2=2 MOD 2 = 0

0+0+0+1+0+0+0+1=2=2 MOD 2 = 0

SubBytes and InvSubBytes

- For 3

Inverse of 03 = 11110110

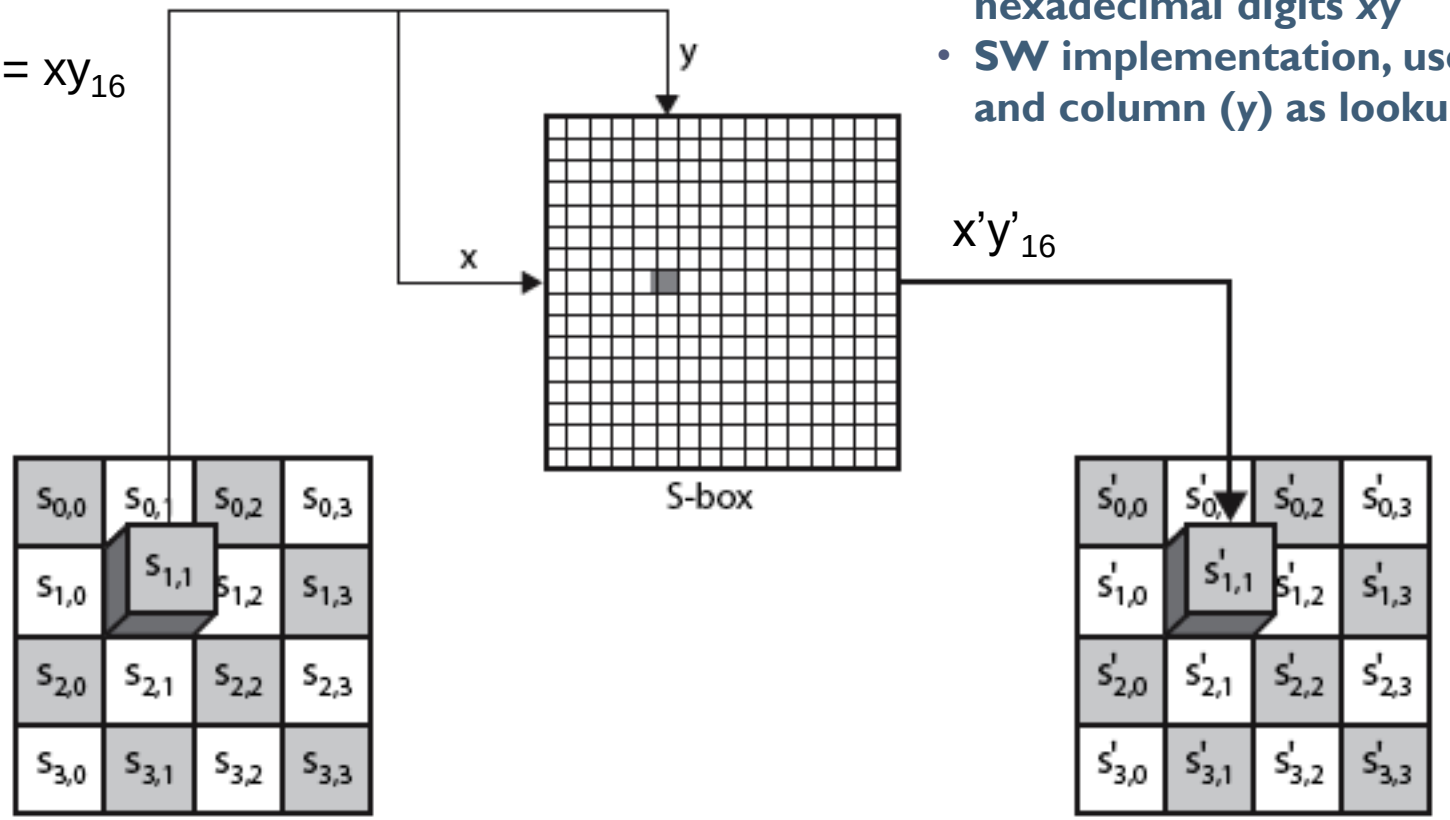


SubBytes Operation

- The SubBytes operation involves 16 independent byte-to-byte transformations.

$S_{1,1} = xy_{16}$

- **Interpret the byte as two hexadecimal digits *xy***
- **SW implementation, use row (*x*) and column (*y*) as lookup pointer**



SubBytes Operation

- Replace each byte in the state array with its corresponding value from the S-Box

55

00	44	88	CC
11	55	99	DD
22	66	AA	EE
33	77	BB	FF

		y														
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16



SubBytes Table

- Implement by Table Lookup

		y														
		0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
x	**0**	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16



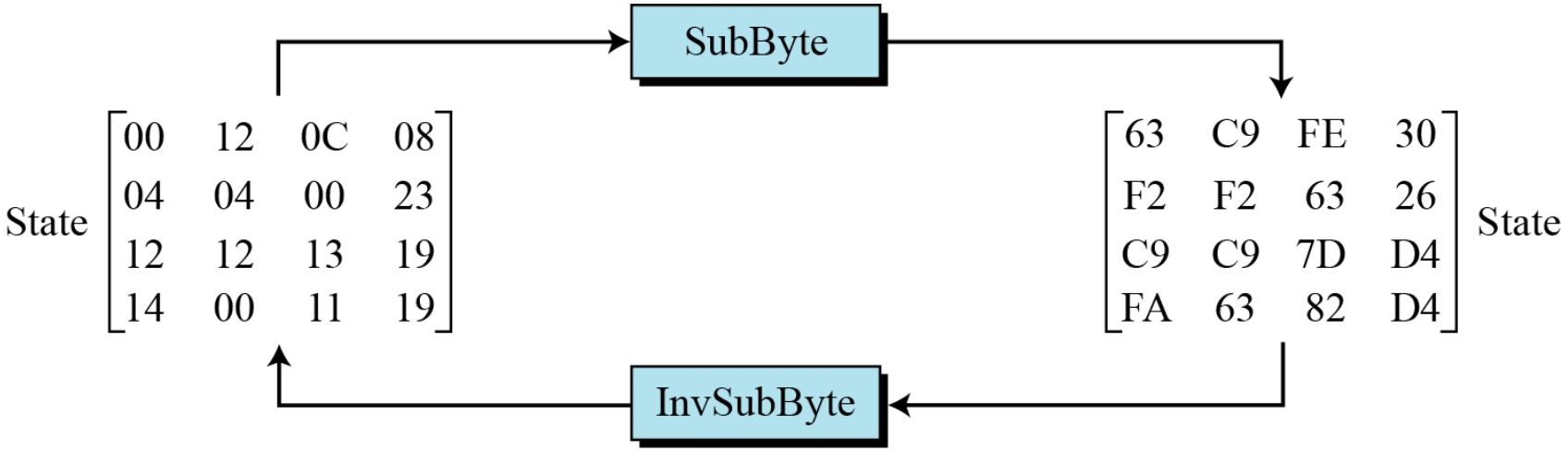
InvSubBytes Table

		y														
		0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
x	**0**	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D



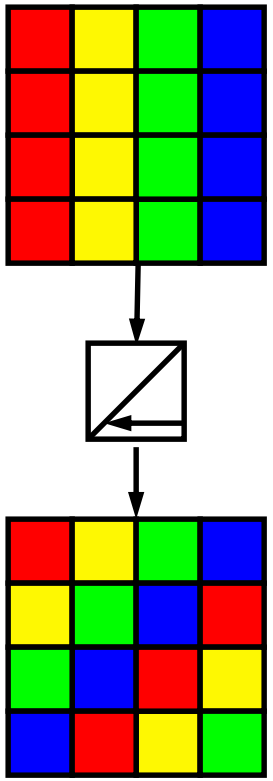
Sample SubByte Transformation

- The SubBytes and InvSubBytes transformations are inverses of each other.



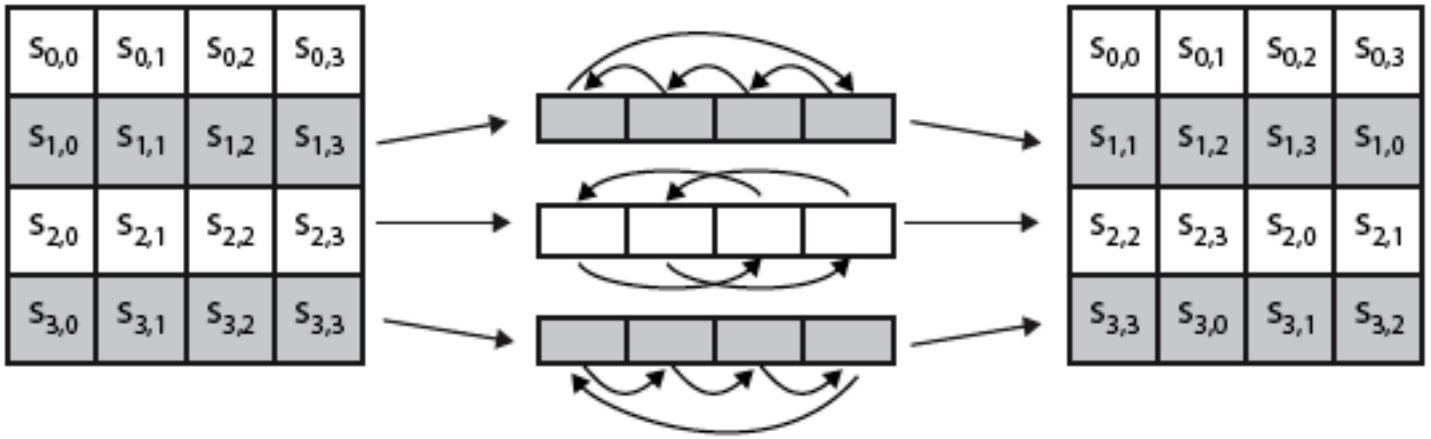
ShiftRows

- Shifting, which permutes the bytes.
- A circular byte shift in each each
  - 1st row is unchanged
  - 2nd row does 1 byte circular shift to left
  - 3rd row does 2 byte circular shift to left
  - 4th row does 3 byte circular shift to left
- In the encryption, the transformation is called ShiftRows
- In the decryption, the transformation is called InvShiftRows and the shifting is to the right

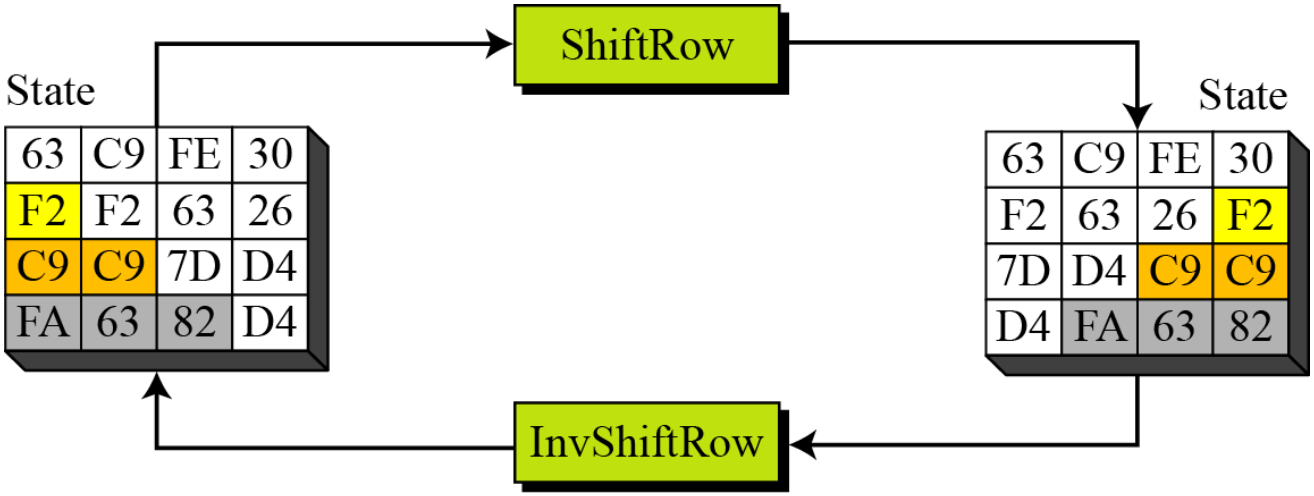




ShiftRows Scheme



ShiftRows and InvShiftRows



Example 2:

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

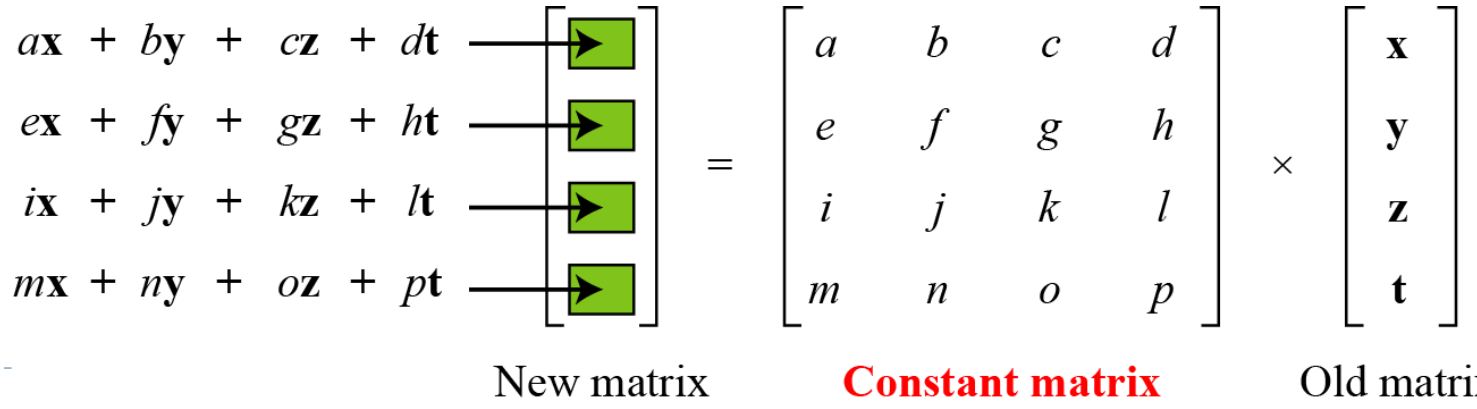
→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

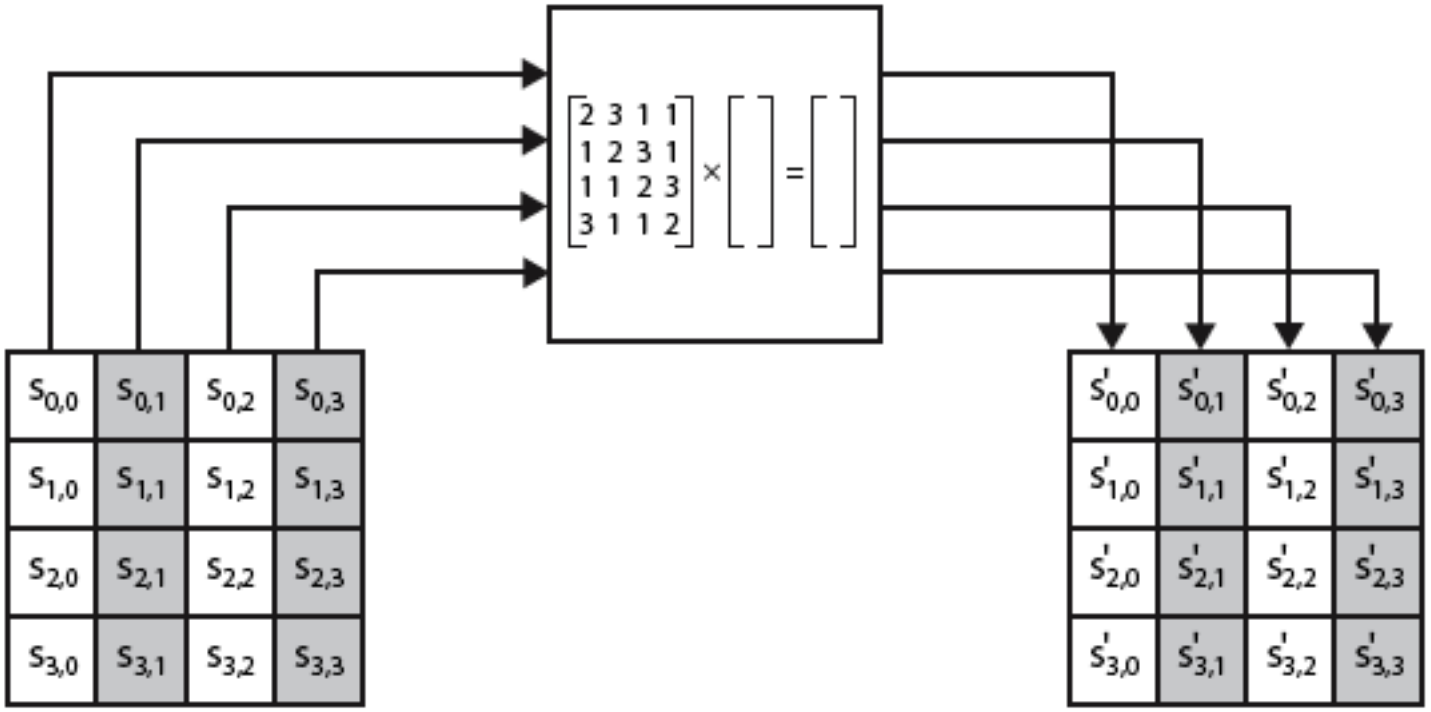


MixColumns

- ShiftRows and MixColumns provide diffusion to the cipher
- Each column is processed separately
- Each byte is replaced by a value dependent on all 4 bytes in the column
- Effectively a matrix multiplication in GF(2^8) using prime poly $m(x) = x^8+x^4+x^3+x+1$



MixClumns Scheme



The MixColumns transformation operates at the column level; it transforms each column of the state to a new column.

MixClumns Scheme

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$



MixClumns Example

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

→

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

$$(\{02\} \bullet \{87\}) \oplus (\{03\} \bullet \{6E\}) \oplus \{46\} \oplus \{A6\} = \{47\}$$

$$\{87\} \oplus (\{02\} \bullet \{6E\}) \oplus (\{03\} \bullet \{46\}) \oplus \{A6\} = \{37\}$$

$$\{87\} \oplus \{6E\} \oplus (\{02\} \bullet \{46\}) \oplus (\{03\} \bullet \{A6\}) = \{94\}$$

$$(\{03\} \bullet \{87\}) \oplus \{6E\} \oplus \{46\} \oplus (\{02\} \bullet \{A6\}) = \{ED\}$$



MixClumns Example

- uses arithmetic in the finite field GF(2^8)
- with irreducible polynomial

$m(x) = x^8 + x^4 + x^3 + x + 1$

which is (100011011) or {11b}

- e.g.

{02} • {87} mod {11b} = (1 0000 1110) mod {11b}

= (1 0000 1110) xor (1 0001 1011) = (0001 0101)



MixColumn and InvMixColumn

